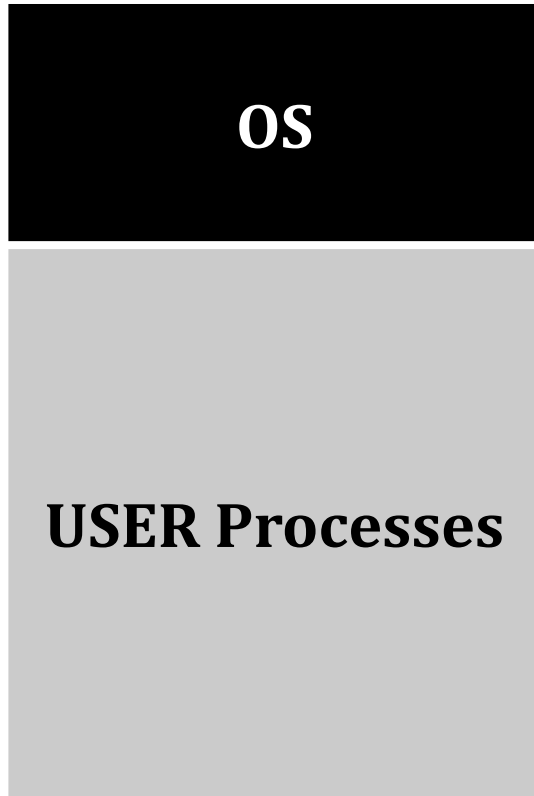


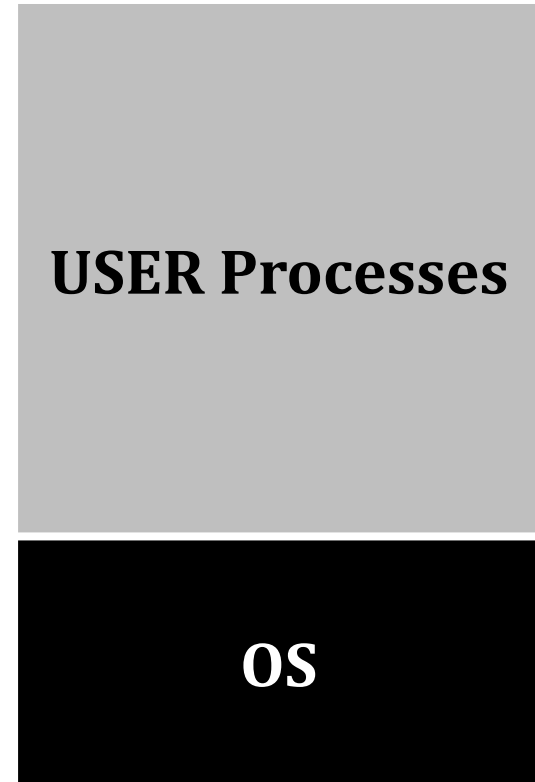
► **Memory Management**

Dr. Manmath N. Sahoo
Dept. of CSE, NIT Rourkela

OS and User Area



or

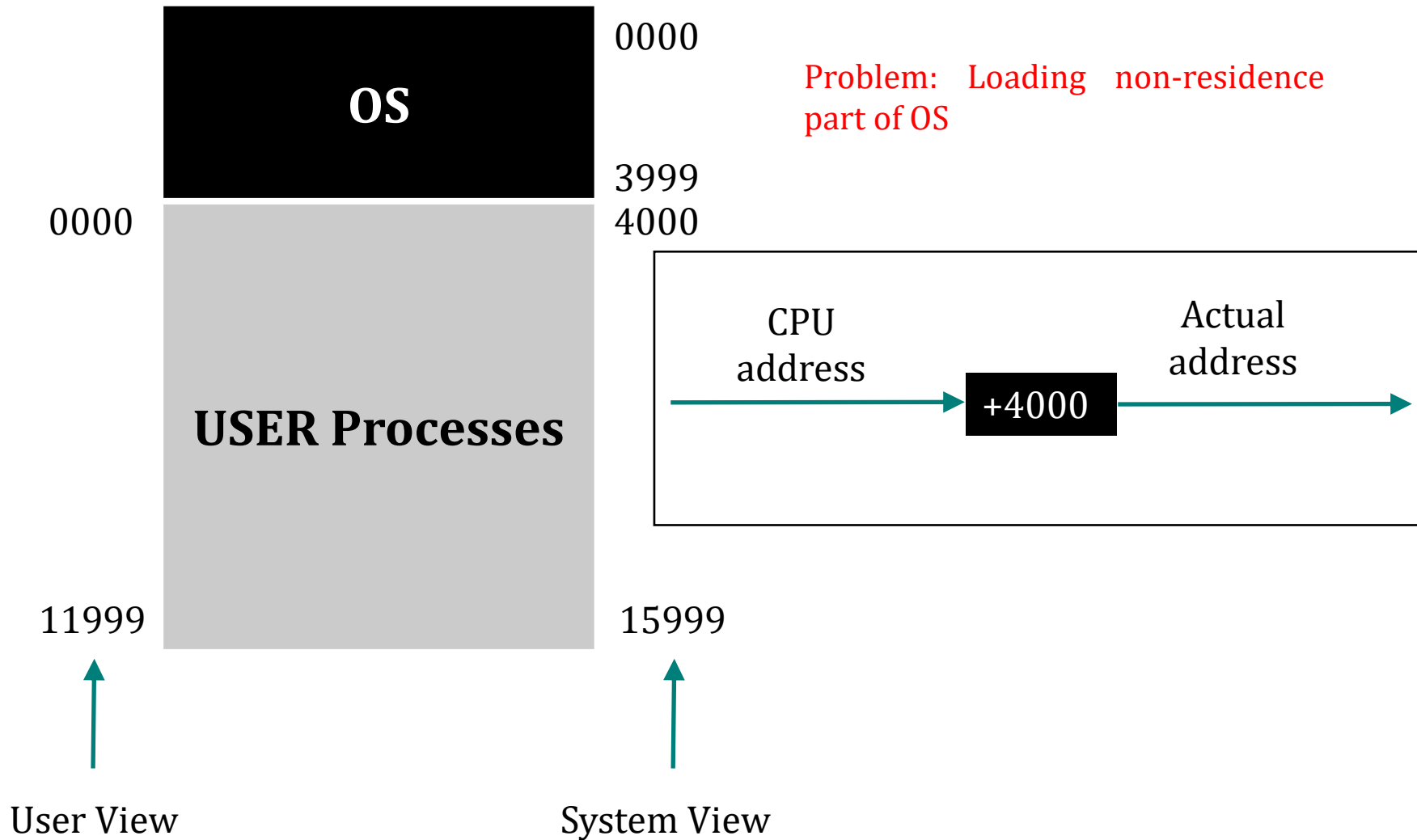


Protection of OS area from users

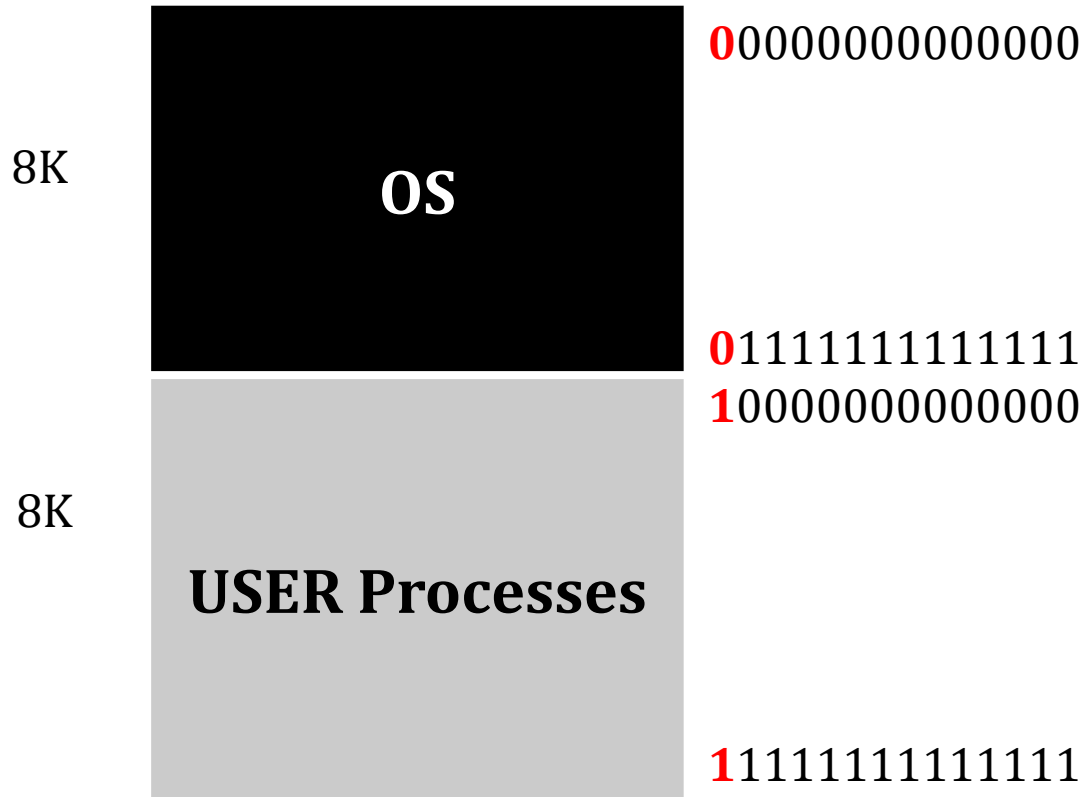
3 Schemes

- ▶ Automatic address translation
- ▶ Page-0 and Page-1 addressing
- ▶ Limit register

Automatic address translation



Page-0 and Page-1 addressing

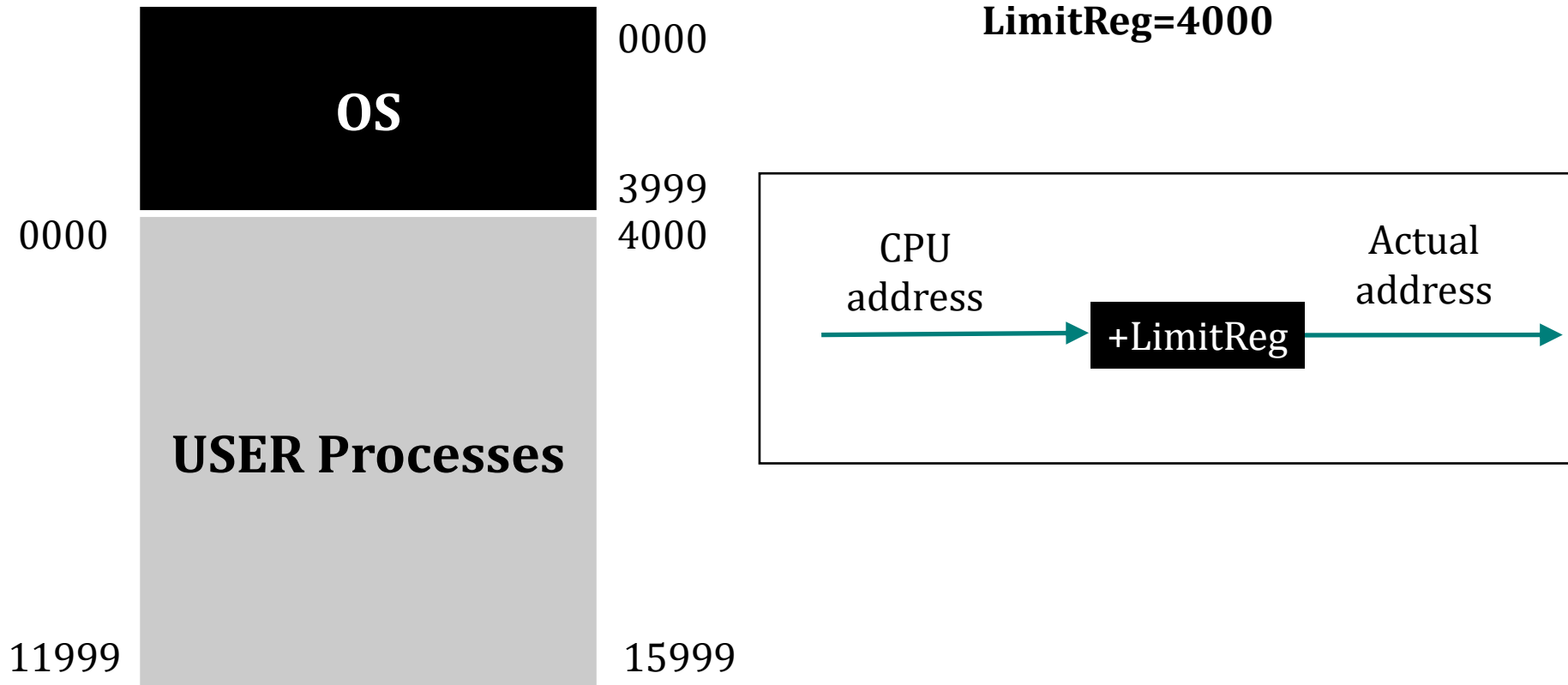


Problems:

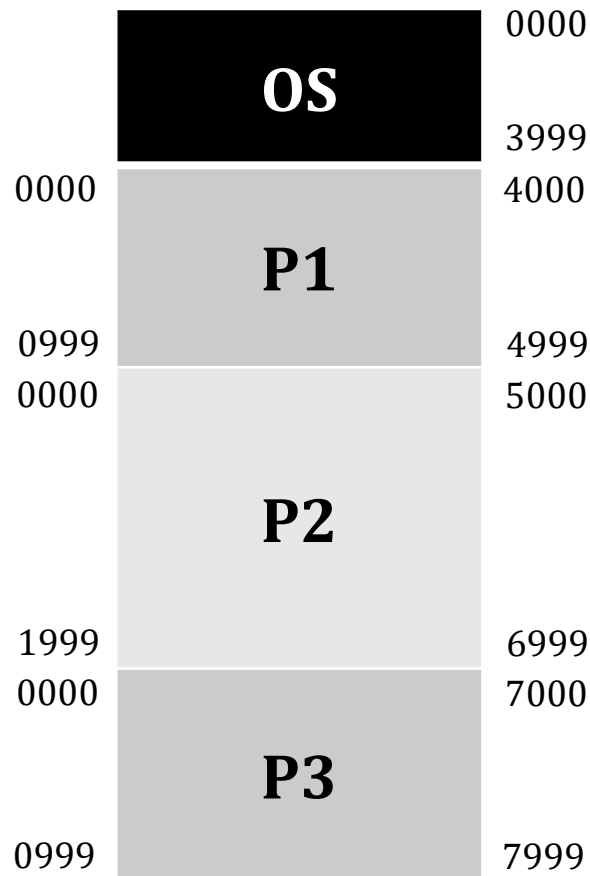
- ▶ Loading non-residence part of OS, may exceed page-0
- ▶ If OS is small then memory wastage

Limit Register

- ▶ Limit register stores the size of the OS



Interprocess protection

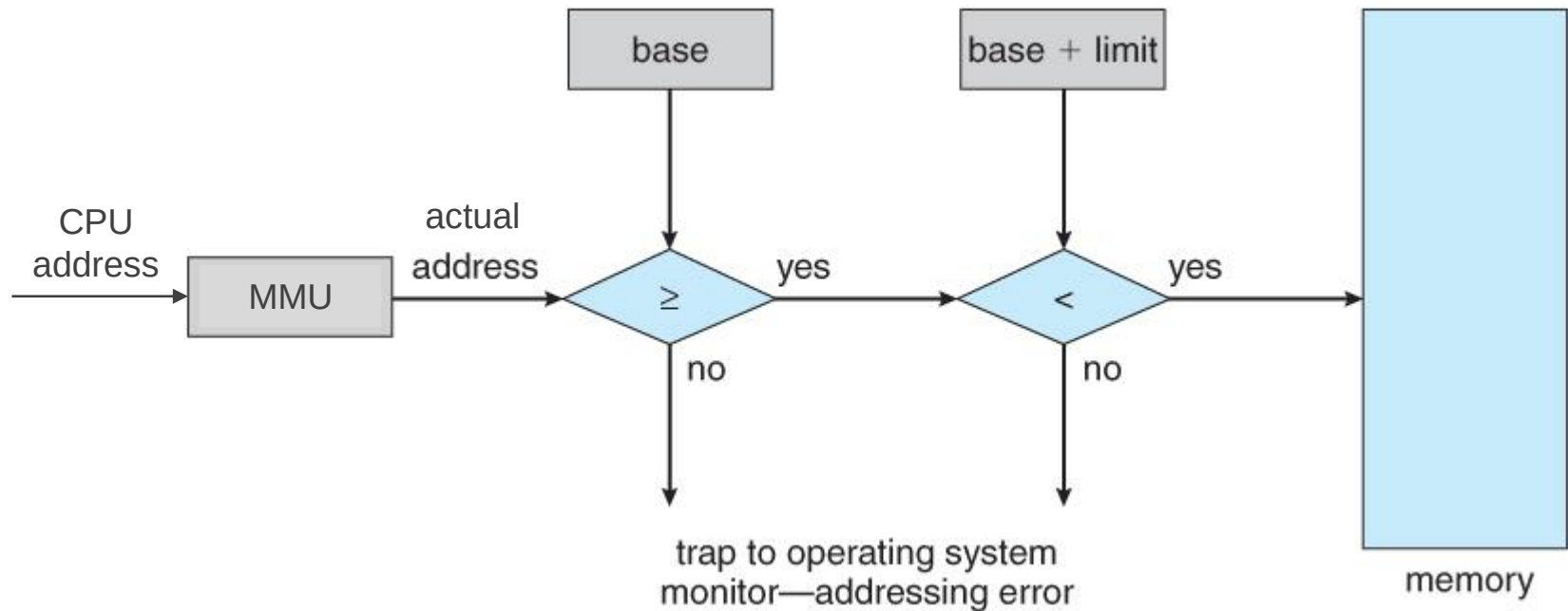


Suppose P2 is running

Base=5000 //base address of P2

Limit=2000 //size of P2

Interprocess protection



Logical address space vs Physical address space

- ▶ CPU generates logical/virtual addresses for instructions and data
- ▶ Logical addresses are converted/mapped to physical addresses by Memory Management Unit (MMU) – hardware unit.

Memory allocation schemes

► Contiguous

- Each process allocated a single contiguous chunk of memory
 - Fixed equal-size partitions
 - Fixed unequal-size partitions
 - Dynamic partitioning

► Noncontiguous

- Parts of a process can be allocated non-contiguous chunks of memory

Fixed equal-size partitions

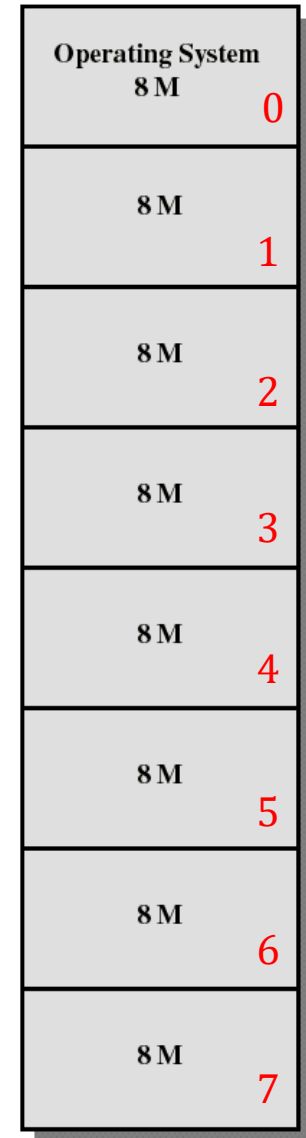
- ▶ If there is an available partition, a process can be loaded into that partition
- ▶ because all partitions are of equal size, it does not matter which partition is used.

MAT

Partition#	Process#
0	OS
1	-
2	-
3	-
4	-
5	-
6	-
7	-

MFT

Partition#	Availability
0	0
1	1
2	1
3	1
4	1
5	1
6	1
7	1



Equal-size partitions

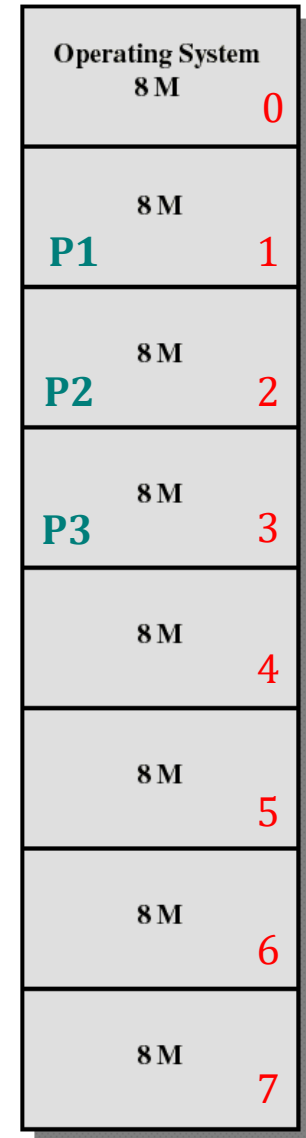
Fixed equal-size partitions

MAT

Partition#	Process#
0	OS
1	P1
2	P2
3	P3
4	-
5	-
6	-
7	-

MFT

Partition#	Availability
0	0
1	0
2	0
3	0
4	1
5	1
6	1
7	1



Equal-size partitions

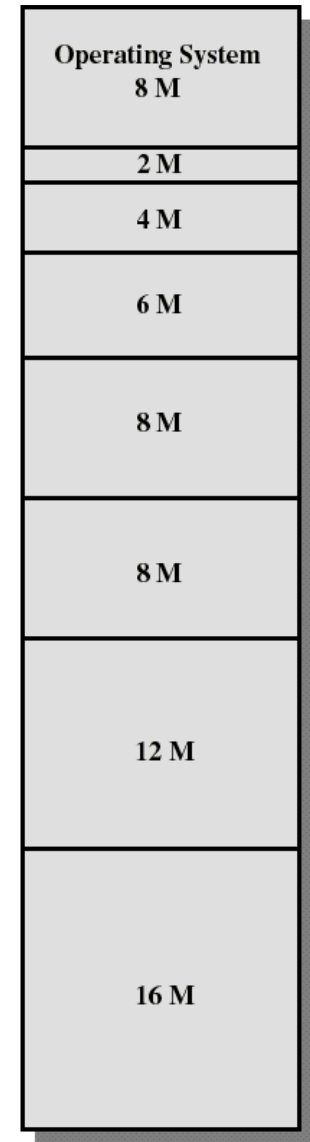
Fixed Unequal-size partitions

MAT

Partition#	Process#
0	OS
1	-
2	-
3	-
4	-
5	-
6	-
7	-

MFT

Partition#	Availability
0	0
1	1
2	1
3	1
4	1
5	1
6	1
7	1

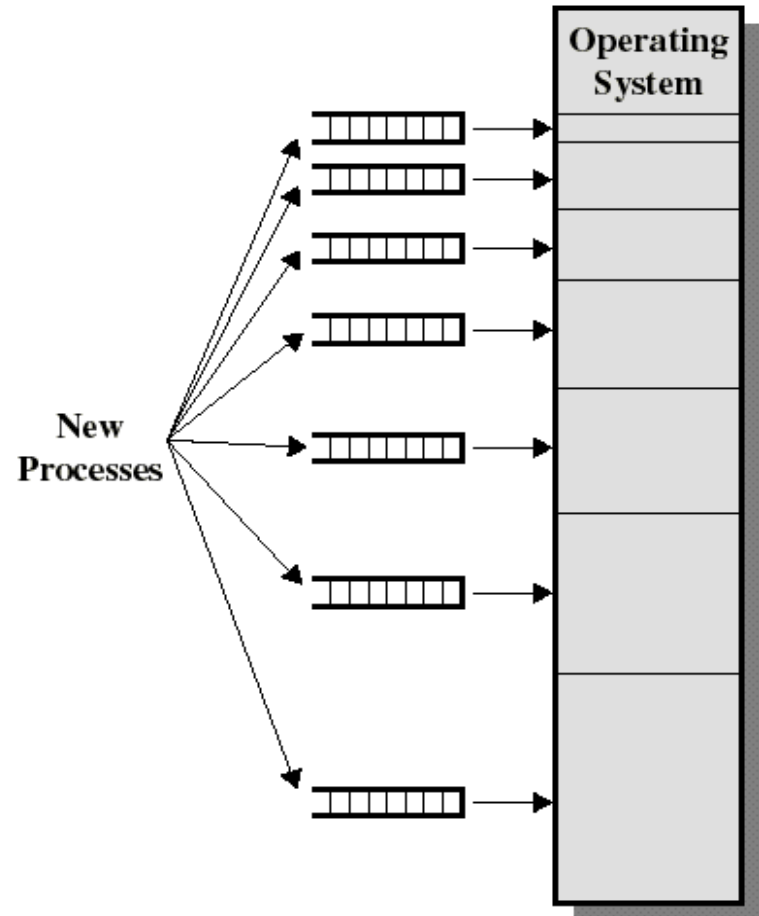


Unequal-size partitions

Placement Algorithm with Unequal-size Partitions

using of multiple queues:

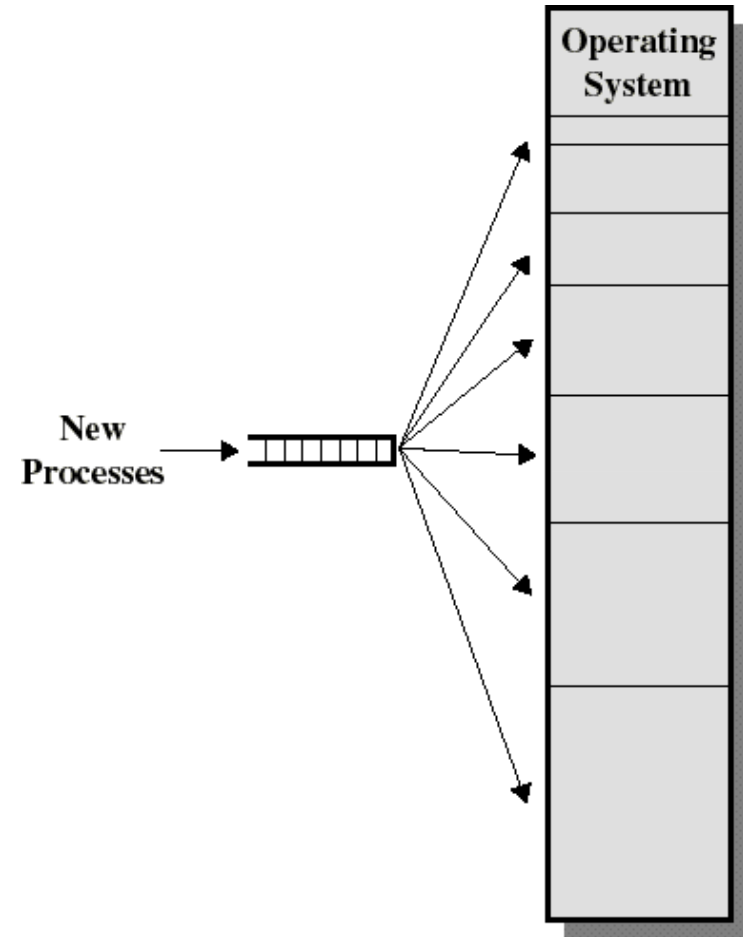
- ▶ a queue exists for each partition size.
- ▶ assign process to the queue of smallest partition within which it can fit.
- ▶ tries to minimize internal fragmentation.
- ▶ problem: some queues might be empty while some might be loaded. – decreases degree of multiprogramming



Placement Algorithm with Unequal-size Partitions

using of a single queue:

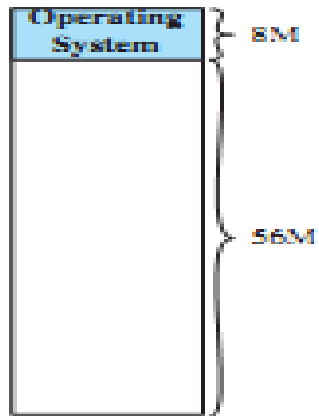
- ▶ the smallest available partition that will hold the process is selected.
- ▶ increases the level of multiprogramming at the expense of internal fragmentation.



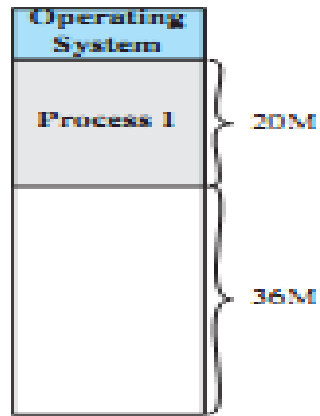
Dynamic partitioning

- ▶ In fixed partitioning scheme, the degree of multiprogramming is limited by number of partitions.
- ▶ **Hole** – a contiguous block of available memory; holes of various size are scattered throughout memory.
- ▶ When a process arrives, it is allocated memory from a hole large enough to accommodate it.
- ▶ Process exiting frees its partition, adjacent free partitions combined.

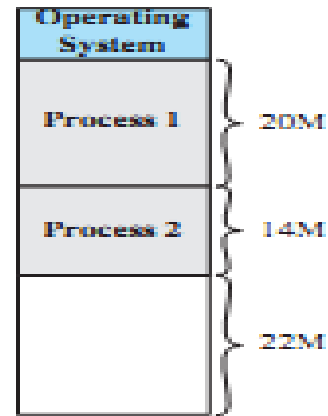
Dynamic partitioning



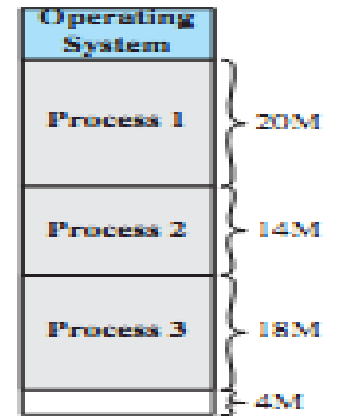
(a)



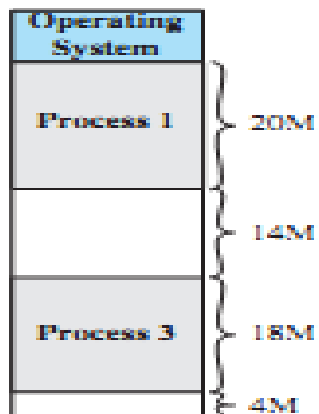
(b)



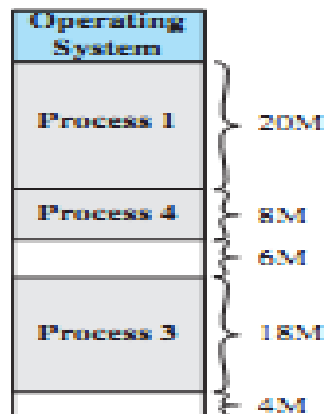
(c)



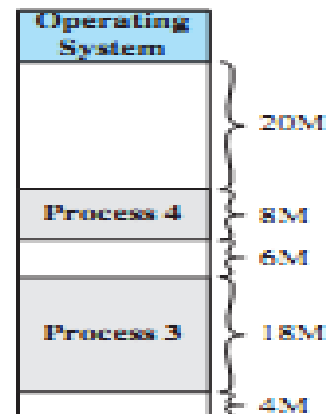
(d)



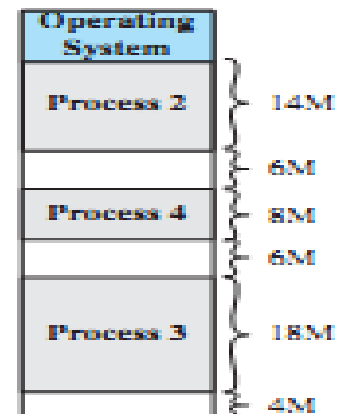
(e)



(f)



(g)



(h)

Dynamic partitioning

MAT

Process#	Starting address	Displacement
----------	------------------	--------------

MFT

Starting address	Displacement
8000	63999

- + No internal fragmentation
- + Degree of multiprogramming is no longer limited by the no. of partitions.
- External fragmentation still exists

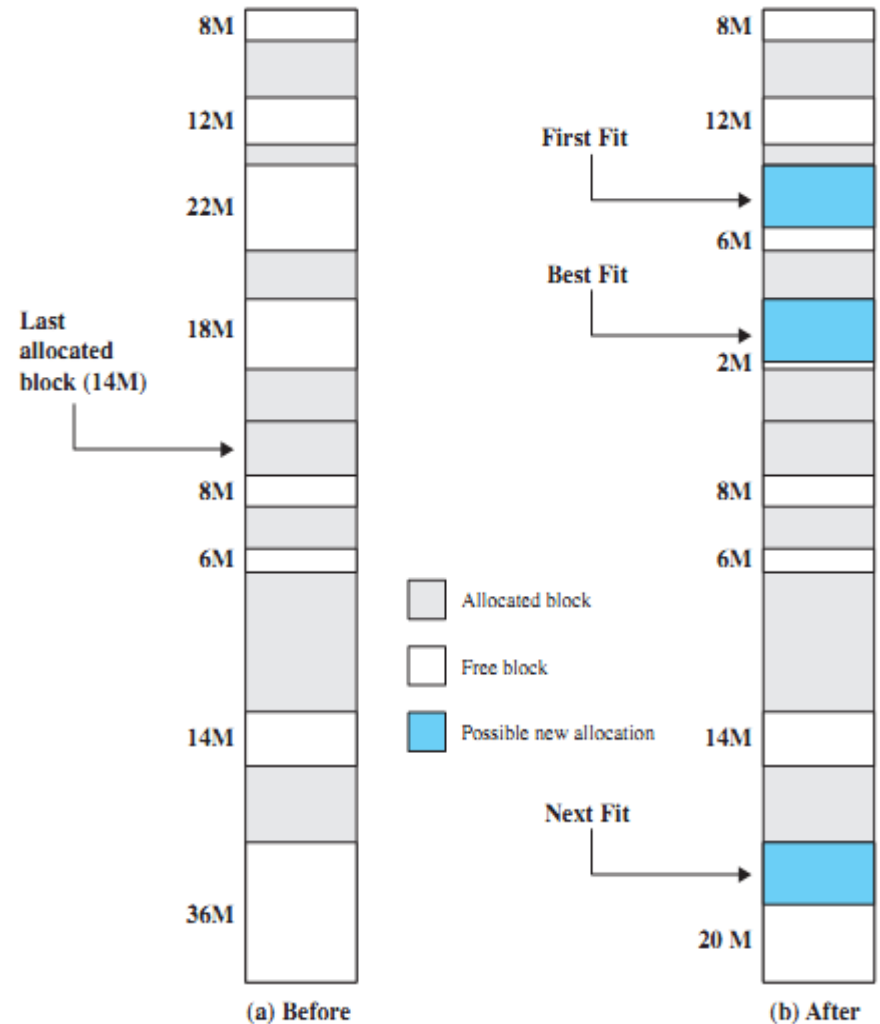
Placement Algorithms in Dynamic partitioning

To satisfy request of size n from list of free holes – four basic methods:

- ▶ **First-fit:** Allocate the first hole that is big enough.
- ▶ **Next-fit:** Same logic as first-fit but starts search always from the last allocated hole (need to keep a pointer to this) in a wraparound fashion.
- ▶ **Best-fit:** Allocate the smallest hole that is big enough; must search entire list, unless ordered by size. Produces the smallest leftover hole.
- ▶ **Worst-fit:** Allocate the largest hole; must also search entire list. Produces the largest leftover hole.

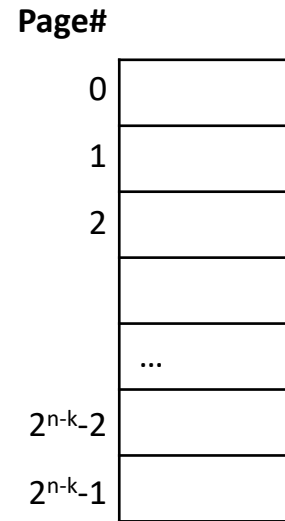
Placement Algorithms in Dynamic partitioning

► New process of 16MB

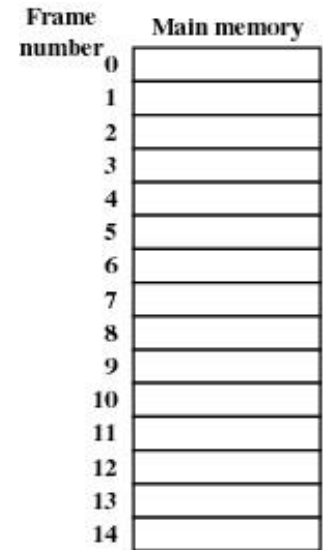


Paging (Noncontiguous allocation)

- ▶ Physical memory is divided up into fixed-sized blocks called **frames**.
- ▶ Logical memory of a process is also divided into blocks of same size as the page frames. These blocks called **pages**.
- ▶ OS keeps track of all free frames.
- ▶ To run a program of size n pages, need to find n free frames and load n pages into them.
- ▶ Then set up a **page table** to translate logical to physical addresses.
- ▶ Separate page table for each process.



(LA Space)



(a) Fifteen Available Frames

2^n : logical addr size

2^k : page size

Page#	Frame#	Valid bit
0	0	1
1	1	1
2	2	1
3	3	1
	-	0
	...	...
$2^{n-k}-1$	-	0

Page Table of Process A with 4 valid pages

Paging

Frame number	Main memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

(a) Fifteen Available Frames

	Main memory
0	A.0
1	A.1
2	A.2
3	A.3
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

(b) Load Process A

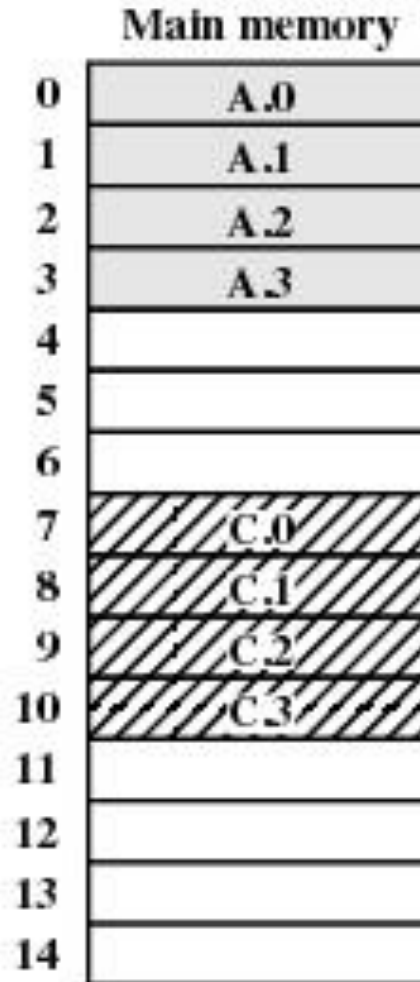
	Main memory
0	A.0
1	A.1
2	A.2
3	A.3
4	B.0
5	B.1
6	B.2
7	
8	
9	
10	
11	
12	
13	
14	

(b) Load Process B

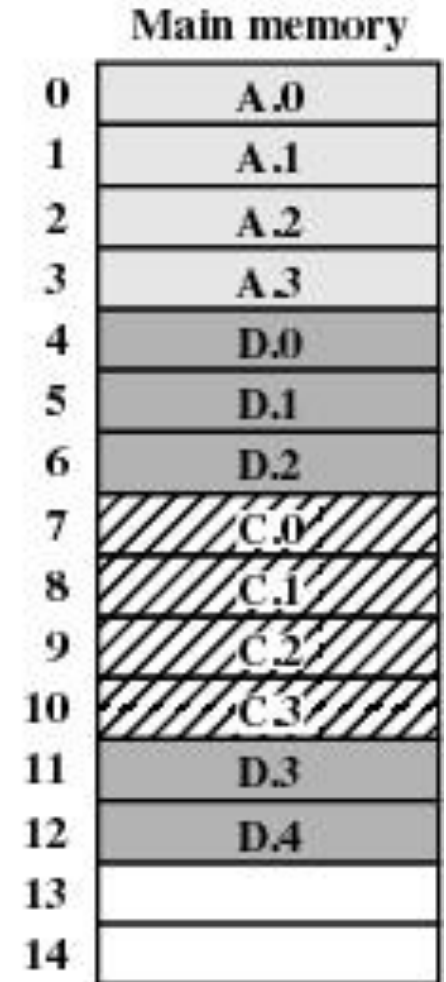
Paging



(d) Load Process C



(e) Swap out B



(f) Load Process D

Page Tables for Example

0	0
1	1
2	2
3	3

Process A
page table

0	—
1	—
2	—

Process B
page table

0	7
1	8
2	9
3	10

Process C
page table

0	4
1	5
2	6
3	11
4	12

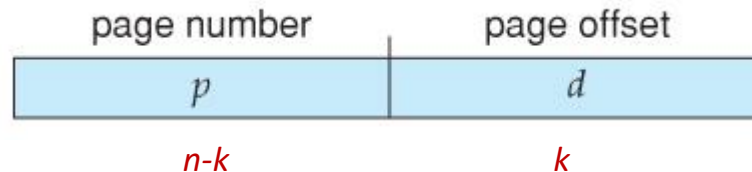
Process D
page table

13
14

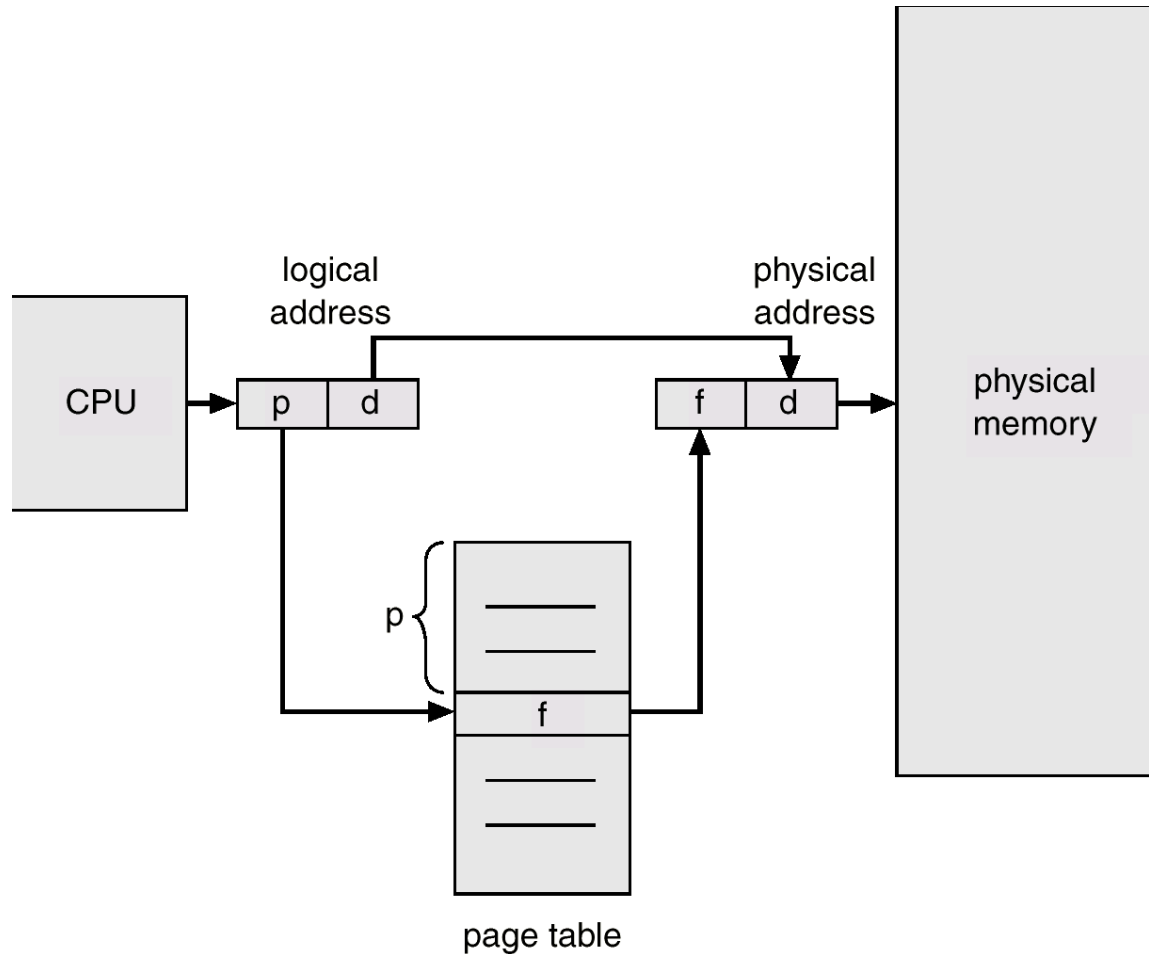
Free frame
list

Address Translation Scheme: Paging

- ▶ Address generated by CPU is divided into:
- ▶ Page number (p) – used as an index into a page table which contains base address of each page in physical memory.
- ▶ Page offset (d) – combined with base address to define the physical memory address that is sent to the memory unit.
- ▶ Logical address space is 2^n , and frame size is 2^k .
 - ▶ Higher order ($n-k$) bits are used for page number (p) and lower order k bits are used for page offset (d)



Address Translation Scheme: Paging



Implementation of Page Table

- ▶ Set of dedicated registers:
 - ▶ Let 16-bit logical address, and page size=8K.
 - ▶ Page table contains 8 entries.
 - ▶ 8 dedicated registers can store the page-frame mappings.
 - ▶ Not practical if page table contains large number of entries.

Implementation of Page Table

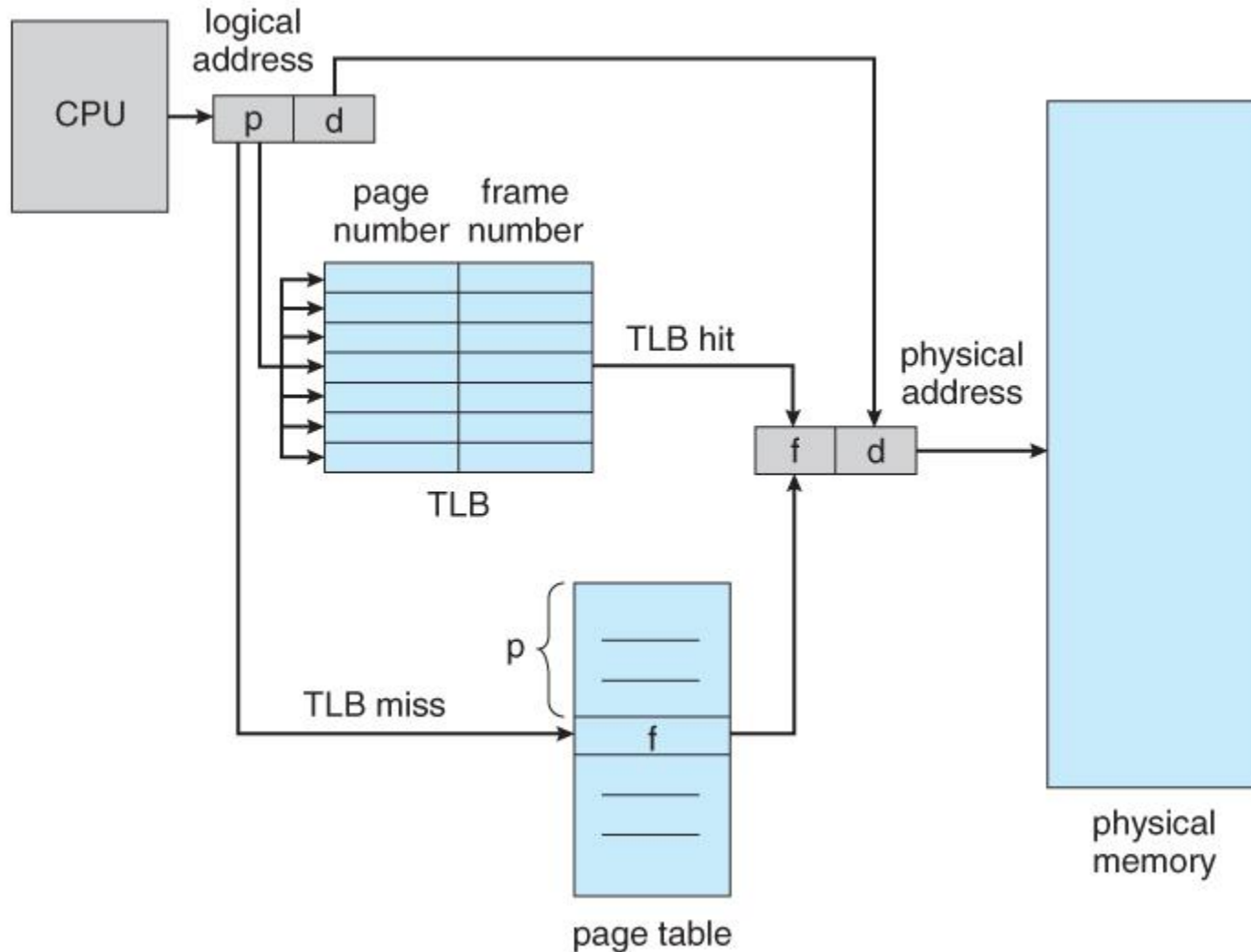
- ▶ PMT kept in main memory.
 - ▶ Page Table Base Register (PTBR) points to the PMT.
 - ▶ Page Table Length Register (PTLR) stores size of the PMT.
 - ▶ Memory access is slowed down by a factor of 2.

Implementation of Page Table

- ▶ PMT kept in main memory and TLB
 - ▶ A small, high speed associative cache called Translation Look-aside Buffer (TLB) stores some entries of PMT.
 - ▶ PMT, as a whole is stored on main memory.
- ▶ TLB (Associative Cache) – allow parallel search

Page#	Frame#

TLB hardware for paging



Effective memory access time

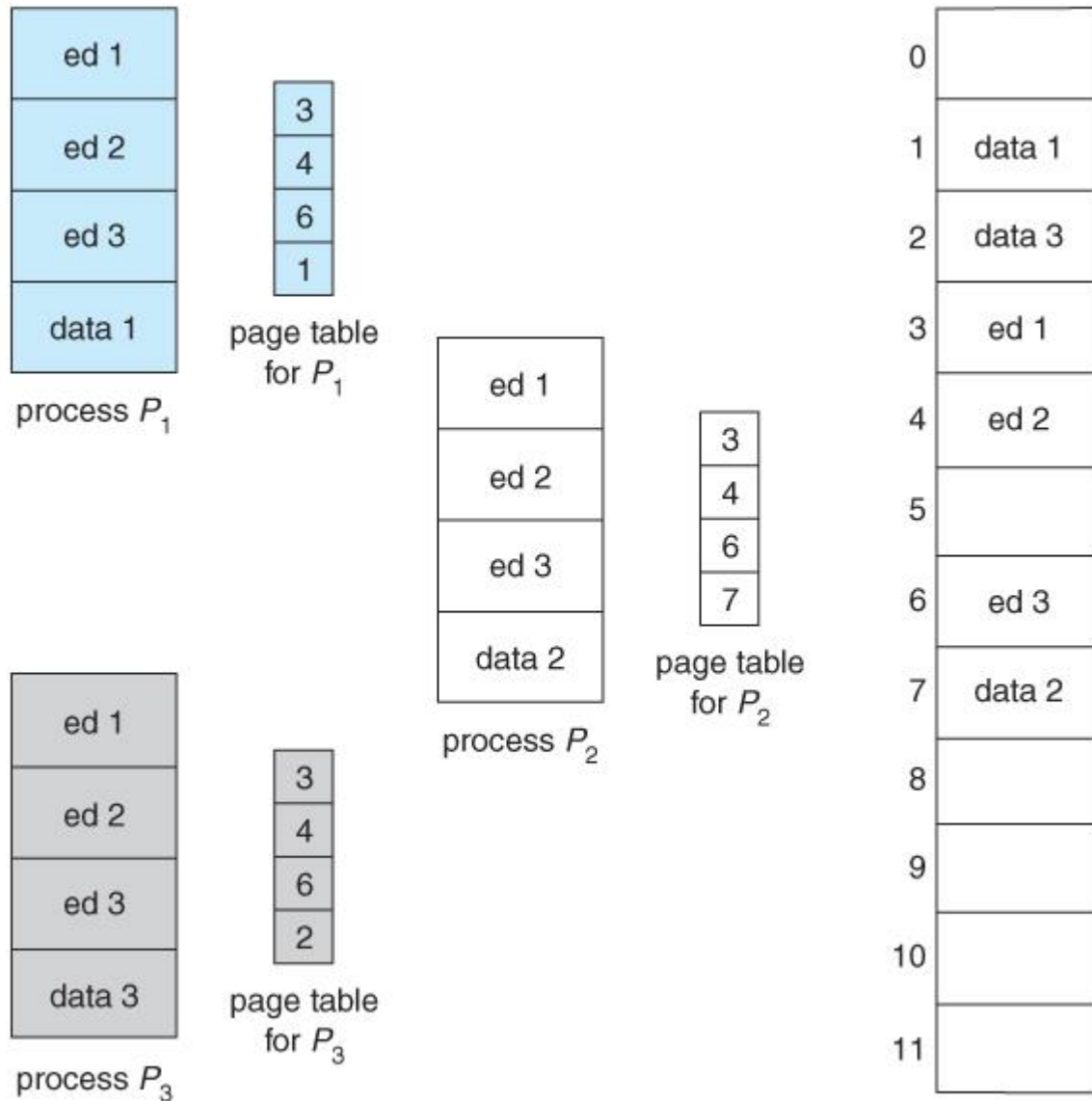
- ▶ Memory access time = 100ns
- ▶ TLB lookup time = 20ns
- ▶ TLB hit ratio = 80%
- ▶ Effective access time =

$$0.8 * (20+100) + 0.2 * (20+2*100) = 140 \text{ ns}$$

Shared Pages

- ▶ Let 40 users executing text editor
- ▶ Text editor: 150KB code + 50KB data
- ▶ Total memory required = 8000KB (more)
- ▶ However, code pages can be shared
- ▶ Let page size = 50KB then 3 code pages and 1 data page per user

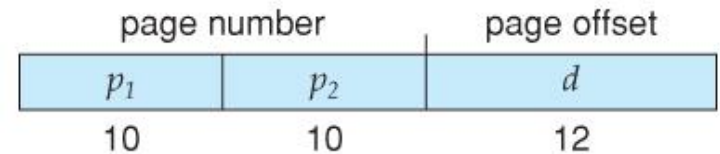
Shared Pages



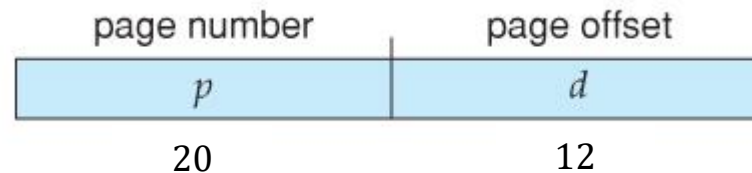
150KB shared code +
40*50KB data

=2150KB (40 users)

Page Table Structure



- ▶ Most modern computer systems support 32bit or 64bit logical address.
- ▶ Consider a system with 32bit logical address, and page size of 4K



- ▶ # of entries in PMT = 2^{20}
- ▶ Each entry of PMT stores a 21bit frame# and other attributes (for a 8GB memory).
- ▶ If each entry is of 4bytes then memory required by PMT = $4 * 2^{20} = 4\text{MB}$ (can't fit into a single frame)
- ▶ Do we need to apply paging on the PMT itself??

YES – Hierarchical Paging

2-level paging example

Page size=3K, each entry in PMT=1K

Process A=18K

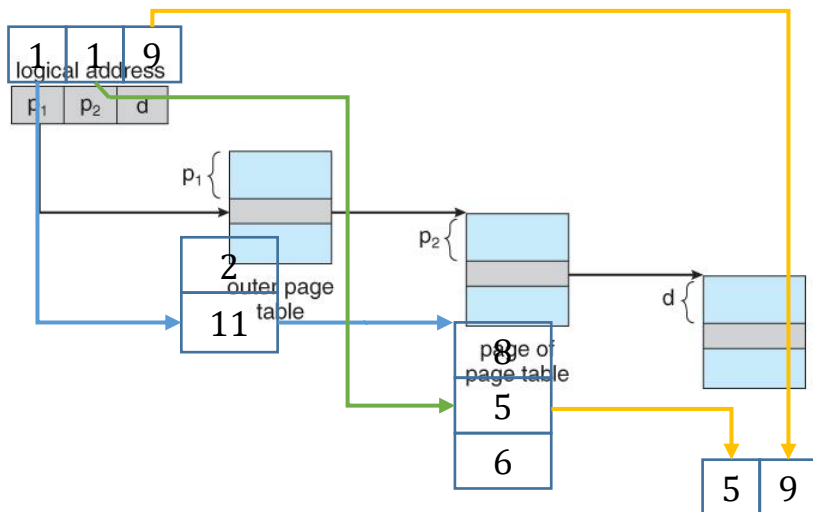
0	
1	
2	
3	
4	
5	

Inner Page Table

0	3	pg0
1	1	
2	7	
3	8	pg1
4	5	
5	6	

Outer Page Table

pg0	2
pg1	11

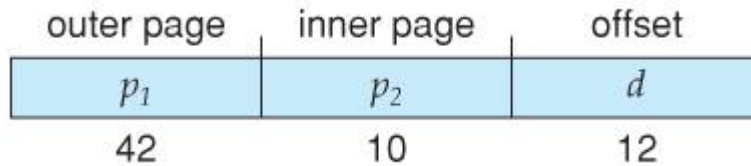


Main memory

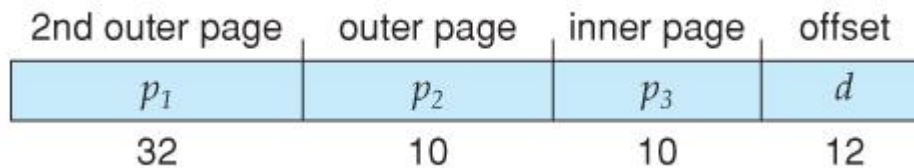
0	
1	A.1
2	pg0
3	A.0
4	
5	A.4
6	A.5
7	A.2
8	A.3
9	
10	
11	pg1
12	
13	
14	

Hierarchical paging

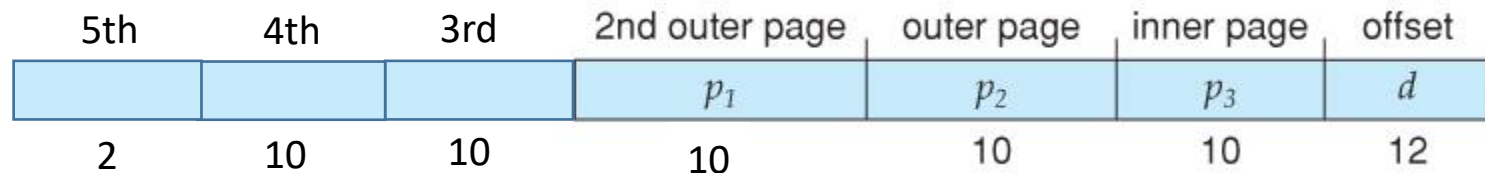
► For a 64-bit logical address system:



64-bits Two-level leaves 42 bits in outer table

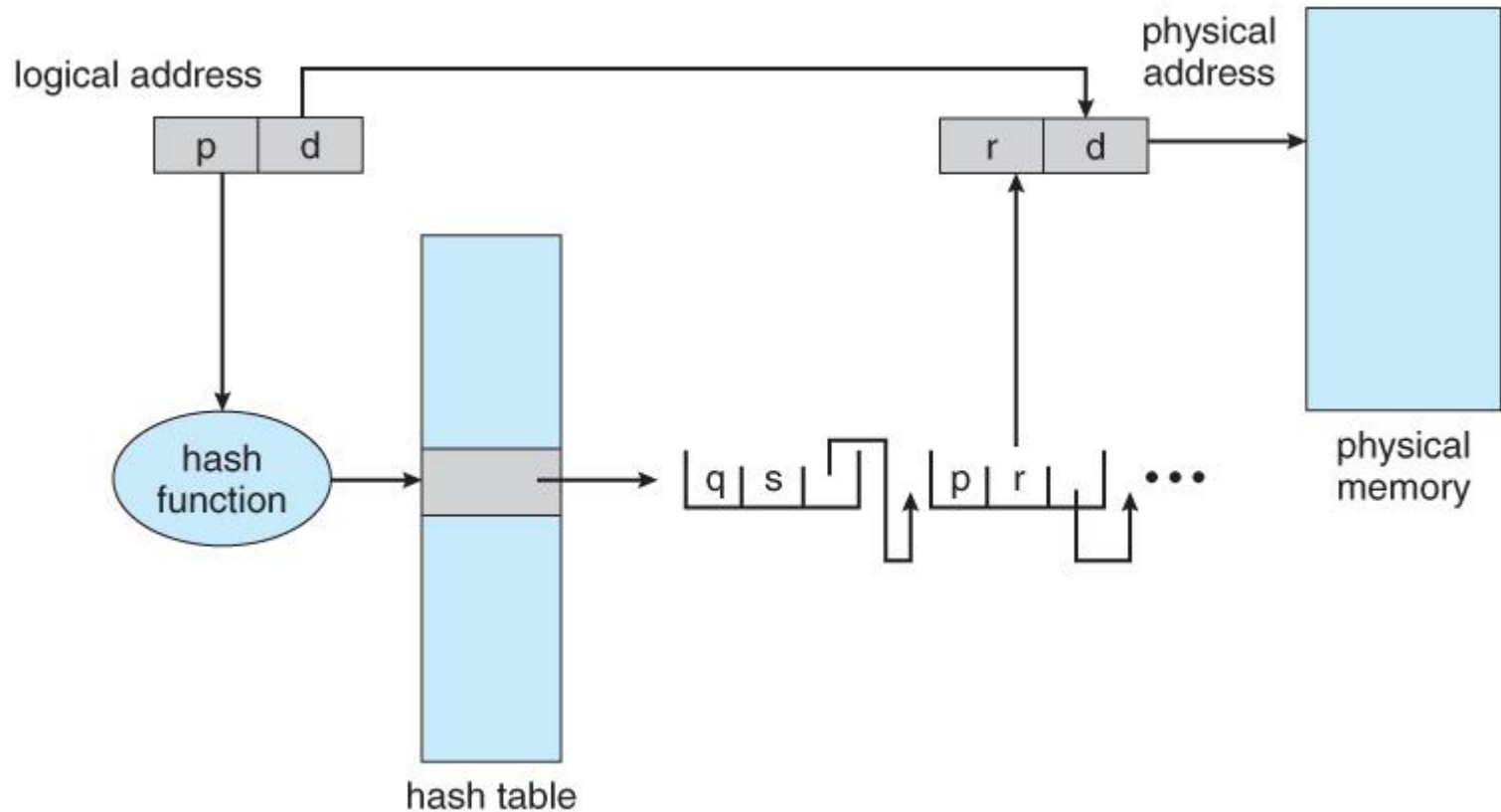


Going to a fourth level still leaves 32 bits in the outer table.



7 levels of indirection, which would be prohibitively slow memory access

Hashed Page Tables



Inverted Page Table

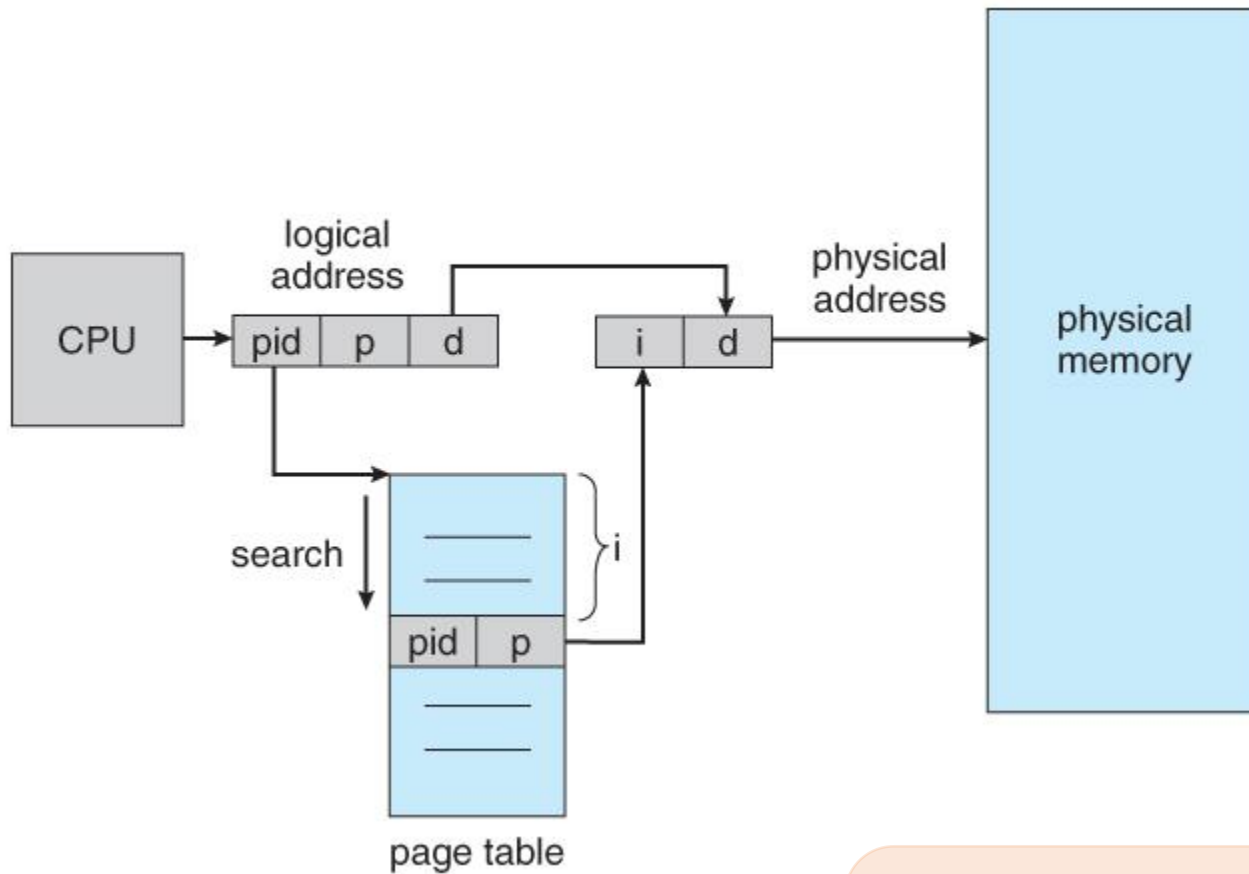
- ▶ Managing per-process PMT is complex.
- ▶ Inverted page table keeps an entry corresponding to each physical frame

A	B	C
0	0	0
1	1	1
2		

Inverted PMT	
0	B 0
1	A 0
2	A 1
3	B 1
4	C 0
5	A 2
6	C 1
7	

Main memory	
0	B.0
1	A.0
2	A.1
3	B.1
4	C.0
5	A.2
6	C.1
7	

Inverted Page Table



Logical Address

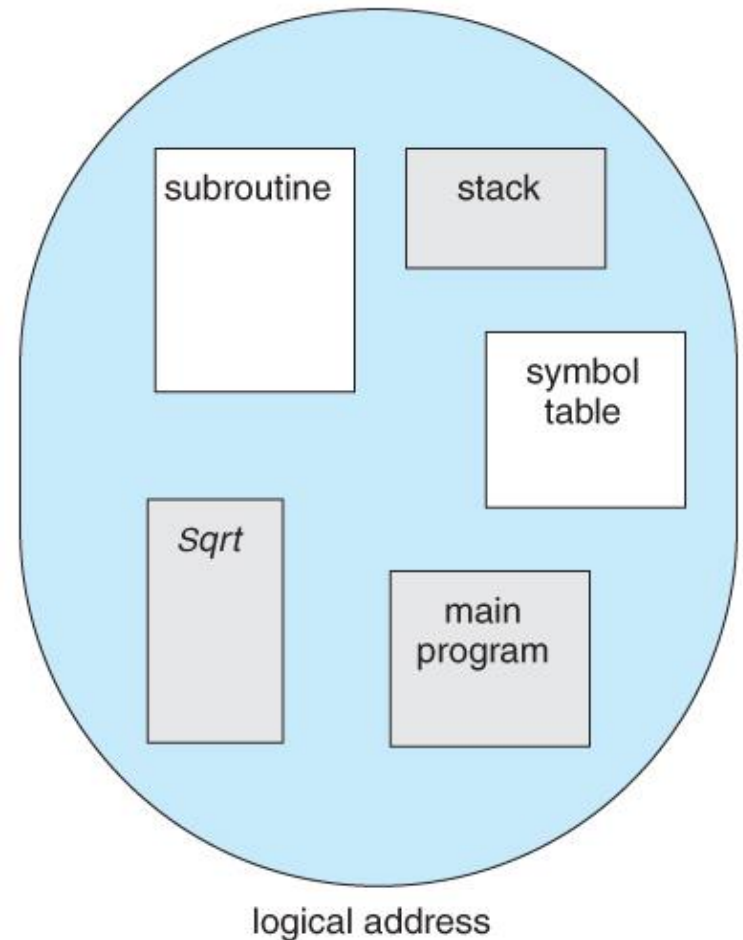
B 1 9

Physical Address

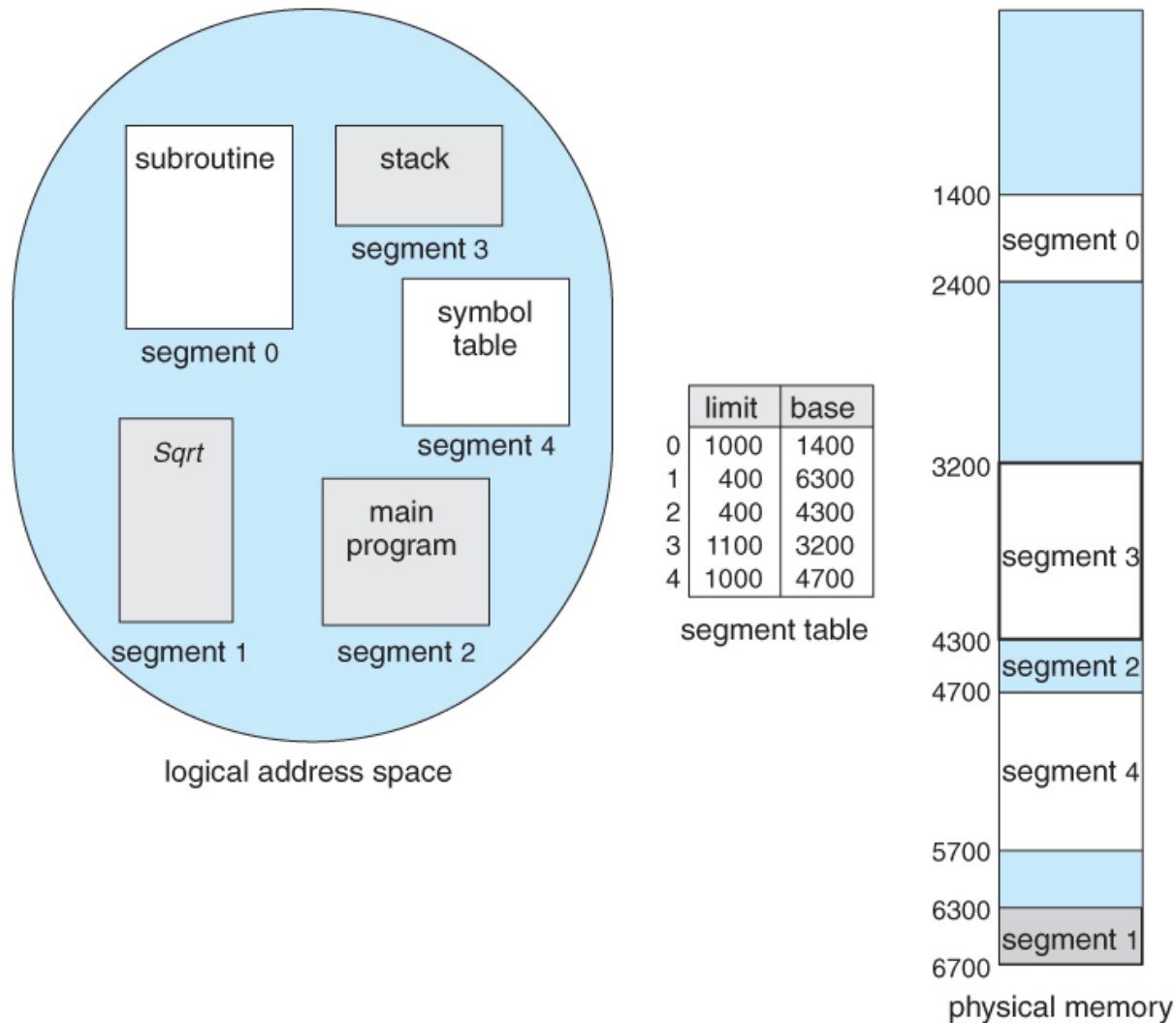
3 9

Segmentation

- ▶ User view (at higher level of abstraction) - Process as a collection of blocks (segments)
- ▶ E.g., `main()`, `sqrt()`, stack etc



Segmentation Example

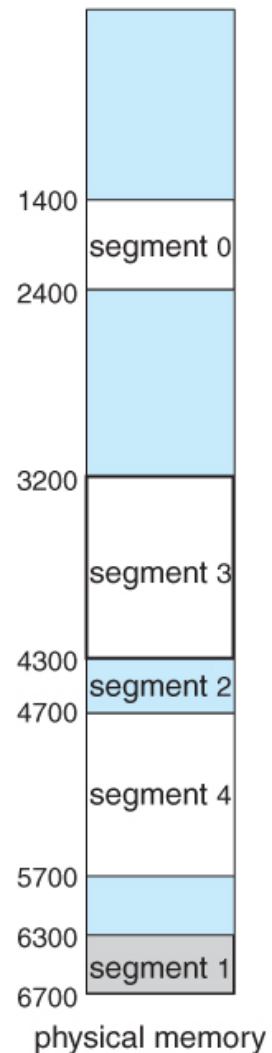


Segmentation Hardware

- ▶ Logical address consists of a pair: $\langle \text{segment-number}, \text{offset} \rangle$,
 - ▶ Segment number is an index into the segment table - with one segment table for each process
 - ▶ offset is the value to be added to start address of segment to give real physical address
- ▶ each entry in segment table has:
 - ▶ base value – contains the starting physical address where the segment resides in memory.
 - ▶ limit – specifies the length of the segment.
- ▶ Offset address is legal if $\text{offset} < \text{limit}$ value

	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

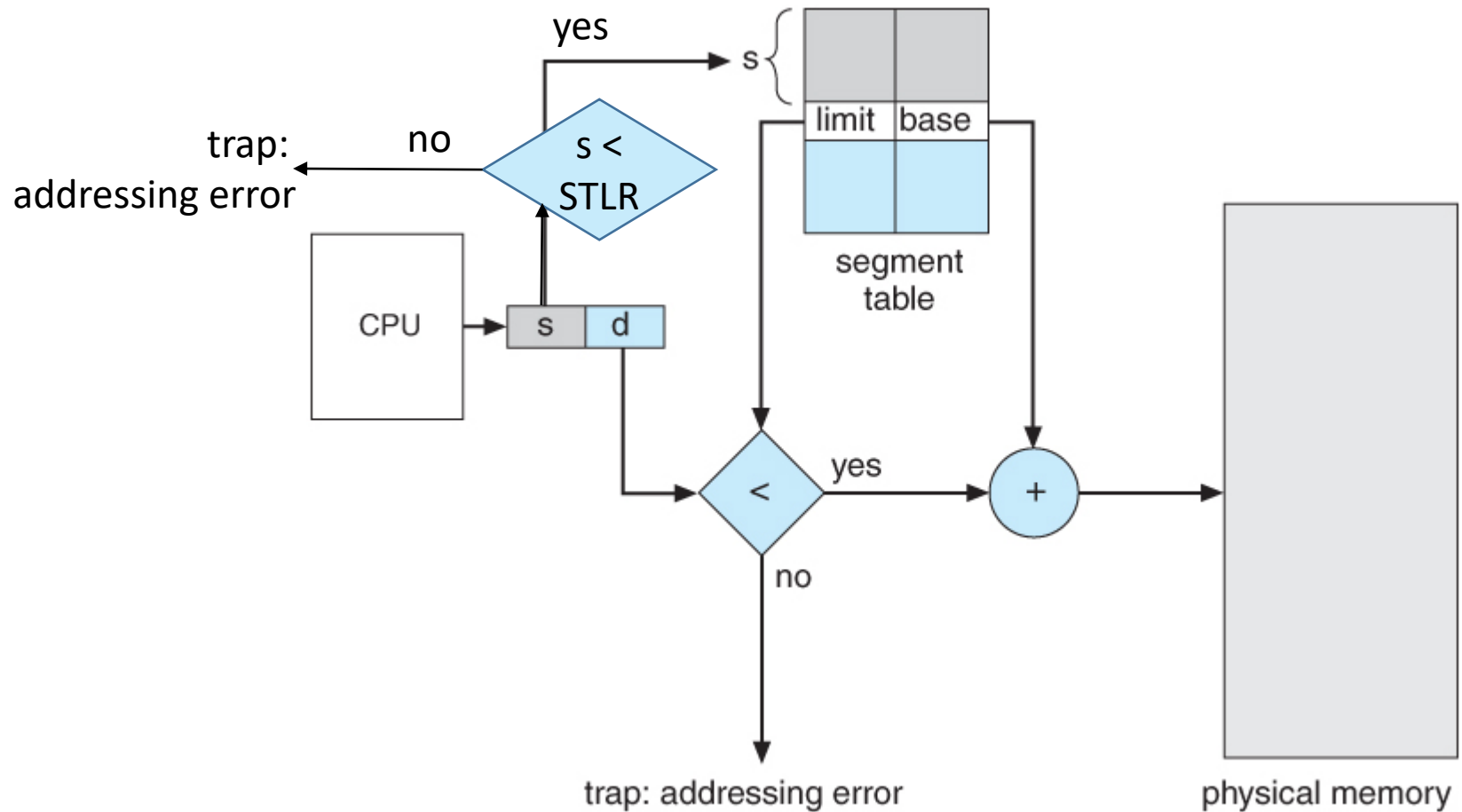
segment table



Segmentation Hardware

- ▶ Segment-table base register (STBR) points to the segment table's location in memory.
- ▶ Segment-table length register (STLR) indicates number of segments used by a process;
- ▶ segment number s is legal if $s < \text{STLR}$.

Segmentation Hardware



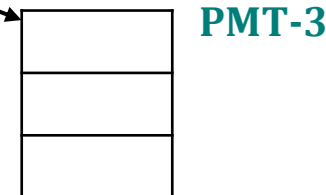
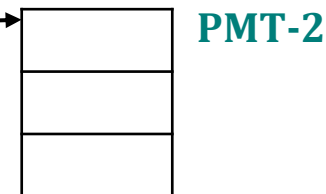
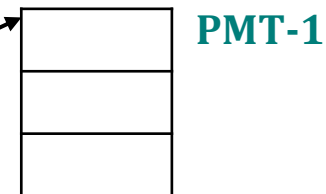
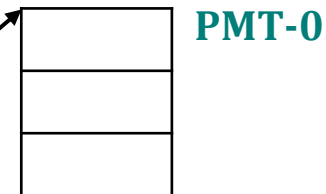
Segmentation with Paging

Logical Address

Segment#	Page#	Offset
----------	-------	--------

Segment Table

Segment#	Page Table Base
0	
1	
2	
3	



Segmentation with Paging: Address Translation

